

Agent 一线实战：落地经验与 Know-How

张开翼 (Kai)

Moonshot AI

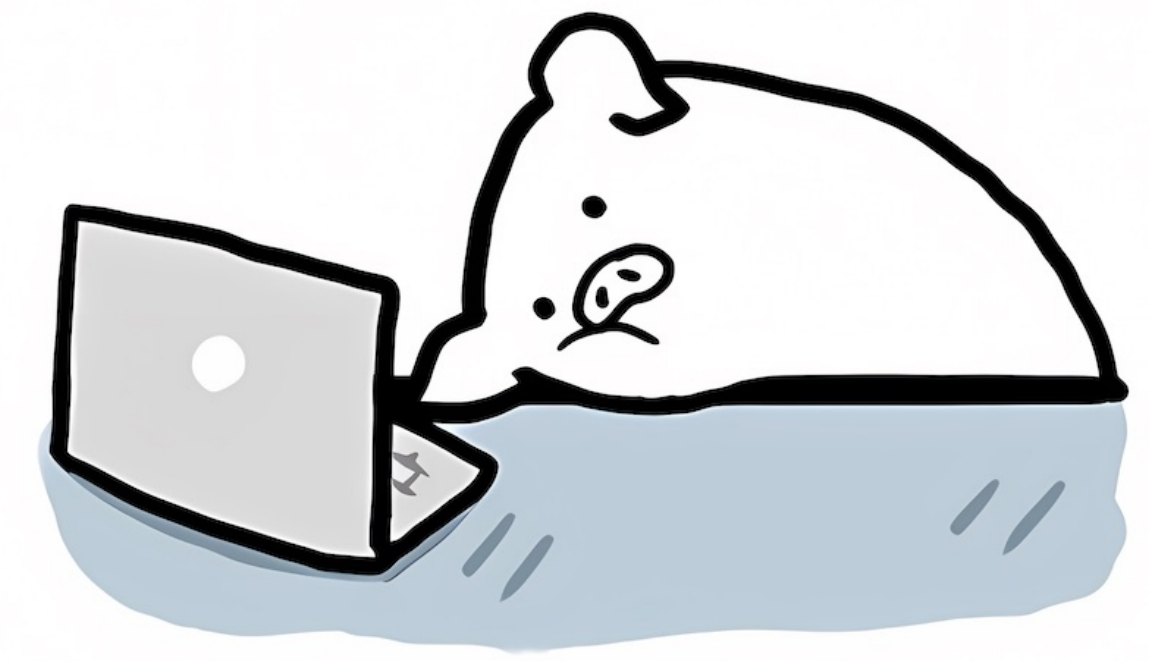
Technical Staff



关于我

- 张开翼 (Kai)
- 月之暗面 (Moonshot AI) Technical Staff
- Kimi Code CLI “主理人”
- 前微软北美工程师

- X: @real_kai42
- Homepage: <https://kaiyi.cool/>



整体框架

- Agent 有价值的是什么？
- Agent 研发经验
- Agent 未来

Agent 框架本身价值较低

Agent 框架本身价值较低

- 类 langchain/langgraph 框架，希望在 llm 和应用之间构建一层抽象，但提供的价值较低

Agent 框架本身价值较低

- 类 langchain/langgraph 框架，希望在 llm 和应用之间构建一层抽象，但提供的价值较低
- Agent 本身就是一个大的 while 循环，研发成本较低，在 vibe coding 成熟下，成本更低。模型看状态 → 决定下一步 → 执行工具或回答 → 循环

Agent 框架本身价值较低

- 类 langchain/langgraph 框架，希望在 llm 和应用之间构建一层抽象，但提供的价值较低
- Agent 本身就是一个大的 while 循环，研发成本较低，在 vibe coding 成熟下，成本更低。模型看状态 → 决定下一步 → 执行工具或回答 → 循环
- LLM 对全世界来说都是新的，OpenAI 和 Anthropic 并不掌握更多

Agent 框架本身价值较低

- 类 langchain/langgraph 框架，希望在 llm 和应用之间构建一层抽象，但提供的价值较低
- Agent 本身就是一个大的 while 循环，研发成本较低，在 vibe coding 成熟下，成本更低。模型看状态 → 决定下一步 → 执行工具或回答 → 循环
- LLM 对全世界来说都是新的，OpenAI 和 Anthropic 并不掌握更多
- 发现问题的价值极高，很多时候“你不知道你不知道”，“不知道模型不知道”

Agent 框架本身价值较低

- 类 langchain/langgraph 框架，希望在 llm 和应用之间构建一层抽象，但提供的价值较低
- Agent 本身就是一个大的 while 循环，研发成本较低，在 vibe coding 成熟下，成本更低。模型看状态 → 决定下一步 → 执行工具或回答 → 循环
- LLM 对全世界来说都是新的，OpenAI 和 Anthropic 并不掌握更多
- 发现问题的价值极高，很多时候“你不知道你不知道”，“不知道模型不知道”
- Agent 研发的壁垒在于 Know-How，在一线 交互/研发/烧 token/用户洞察 获得的 insights

Context

Context

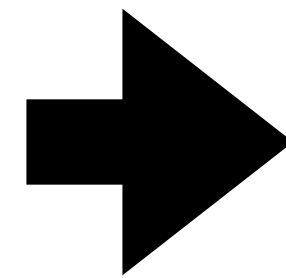
- Context 在可见的未来依旧是一切的核心，“你准备如何构造你的上下文”

Context

- Context 在可见的未来依旧是一切的核心，“你准备如何构造你的上下文”
- 一切都是上下文

OpenAI Harmony

```
{  
  system: "Axxx",  
  developer: "Bxxx",  
  user: "Cxxxx",  
  tools: [{  
    name: "Dxxx",  
    description: "Exxx",  
    parameters: "Fxxx"  
  }]  
}
```



```
<|start|>system<|message|>Axxx<|end|>
```

```
<|start|>developer<|message|>Bxxx
```

```
# Tools
```

```
## functions
```

```
namespace functions {
```

```
// Exxx
```

```
type Dxxx = (_: Fxxx) => any;
```

```
} // namespace functions<|end|>
```

```
<|start|>user<|message|>Cxxxx<|end|>
```

Context

- Context 在可见的未来依旧是一切的核心，“你准备如何构造你的上下文”

Context

- Context 在可见的未来依旧是一切的核心，“你准备如何构造你的上下文”
- 一切都是上下文, 包括 函数、图片、声音 等等一切你的 api 输入

Context

- Context 在可见的未来依旧是一切的核心，“你准备如何构造你的上下文”
- 一切都是上下文, 包括 函数、图片、声音 等等一切你的 api 输入
- Cache 命中, 降低 token 成本、减少延迟

Context

- Context 在可见的未来依旧是一切的核心，“你准备如何构造你的上下文”
- 一切都是上下文, 包括 函数、图片、声音 等等一切你的 api 输入
- Cache 命中, 降低 token 成本、减少延迟
 - system prompt 要稳定, 不要把 时间、环境等可能随时改变的动态信息放入

Context

- Context 在可见的未来依旧是一切的核心，“你准备如何构造你的上下文”
- 一切都是上下文, 包括 函数、图片、声音 等等一切你的 api 输入
- Cache 命中, 降低 token 成本、减少延迟
 - system prompt 要稳定, 不要把 时间、环境等可能随时改变的动态信息放入
 - 不同模型的 follow instruction 的能力和擅长的点不一样, 切换基座模型的成本并不低

Context

- Context 在可见的未来依旧是一切的核心，“你准备如何构造你的上下文”
- 一切都是上下文, 包括 函数、图片、声音 等等一切你的 api 输入
- Cache 命中, 降低 token 成本、减少延迟
 - system prompt 要稳定, 不要把 时间、环境等可能随时改变的动态信息放入
 - 不同模型的 follow instruction 的能力和擅长的点不一样, 切换基座模型的成本并不低
- 随着 Context 的变长, 模型能力会下滑

Context

- Context 在可见的未来依旧是一切的核心，“你准备如何构造你的上下文”
- 一切都是上下文, 包括 函数、图片、声音 等等一切你的 api 输入
- Cache 命中, 降低 token 成本、减少延迟
 - system prompt 要稳定, 不要把 时间、环境等可能随时改变的动态信息放入
 - 不同模型的 follow instruction 的能力和擅长的点不一样, 切换基座模型的成本并不低
- 随着 Context 的变长, 模型能力会下滑
- 上下文的内容对整个项目的影响是很 treaky 的, 渐进式披露/发现

Context

Context

- 错误记录要不要删除？

Context

- 错误记录要不要删除？
- 动态 prompt 注入机制

Context

- 错误记录要不要删除？
- 动态 prompt 注入机制
- prompt 扰动机制

Context

- 错误记录要不要删除？
- 动态 prompt 注入机制
- prompt 扰动机制
- 压缩一定会产生幻觉，幻觉会在长任务中累计，最终失败

Context

- 错误记录要不要删除？
- 动态 prompt 注入机制
- prompt 扰动机制
- 压缩一定会产生幻觉，幻觉会在长任务中累计，最终失败
- Agent team VS Sub Agent

Bitter Lesson for Agent

<https://www.incompleteideas.net/IncIdeas/BitterLesson.html>

Bitter Lesson for Agent

- 基座模型的能力边界影响 Agent 架构

<https://www.incompleteideas.net/IncIdeas/BitterLesson.html>

Bitter Lesson for Agent

- 基座模型的能力边界影响 Agent 架构
- 人类的预定义范式要适时的进入/退出

Bitter Lesson for Agent

- 基座模型的能力边界影响 Agent 架构
- 人类的预定义范式要适时的进入/退出
- 不靠大量硬编码规则和特例，而靠搜索、试错、反馈闭环

Bitter Lesson for Agent

- 基座模型的能力边界影响 Agent 架构
- 人类的预定义范式要适时的进入/退出
- 不靠大量硬编码规则和特例，而靠搜索、试错、反馈闭环
- 架构简洁，不断调整最核心的策略，方便模型理解

Bitter Lesson for Agent

- 基座模型的能力边界影响 Agent 架构
- 人类的预定义范式要适时的进入/退出
- 不靠大量硬编码规则和特例，而靠搜索、试错、反馈闭环
- 架构简洁，不断调整最核心的策略，方便模型理解
- 最核心的依旧是 ReAct 中的，观测 -> 思考 -> 执行 -> 观测

<https://www.incompleteideas.net/IncIdeas/BitterLesson.html>

Tools

Tools

- tool 是占上下文的，tool 也是文本，没有 magic

Tools

- tool 是占上下文的，tool 也是文本，没有 magic
- tools 设计不是给开发者，而是给模型的，命名要有逻辑

Tools

- tool 是占上下文的，tool 也是文本，没有 magic
- tools 设计不是给开发者，而是给模型的，命名要有逻辑
- 基座对 tool 的调用进行过特殊优化，优先在 tool 中实现自己的工具调用，或者利用其进行“magic”

Tools

- tool 是占上下文的，tool 也是文本，没有 magic
- tools 设计不是给开发者，而是给模型的，命名要有逻辑
- 基座对 tool 的调用进行过特殊优化，优先在 tool 中实现自己的工具调用，或者利用其进行“magic”
- tools 不要标新立异，考虑“人类通识性的事实标准”

Tools

- tool 是占上下文的，tool 也是文本，没有 magic
- tools 设计不是给开发者，而是给模型的，命名要有逻辑
- 基座对 tool 的调用进行过特殊优化，优先在 tool 中实现自己的工具调用，或者利用其进行“magic”
- tools 不要标新立异，考虑“人类通识性的事实标准”
- tools 的数量要控制

Tools

- tool 是占上下文的，tool 也是文本，没有 magic
- tools 设计不是给开发者，而是给模型的，命名要有逻辑
- 基座对 tool 的调用进行过特殊优化，优先在 tool 中实现自己的工具调用，或者利用其进行“magic”
- tools 不要标新立异，考虑“人类通识性的事实标准”
- tools 的数量要控制
- tools 不会是未来，mcp 不会是，渐进式披露的 cli 会是

Memory

Memory

- claude code 在最近才开始支持 memory

Memory

- claude code 在最近才开始支持 memory
- 模型并不知道什么是应该记住的，也很难正确提取关键点

Memory

- claude code 在最近才开始支持 memory
- 模型并不知道什么是应该记住的，也很难正确提取关键点
- memory 如何存储，如何分层次存储，如何检索，如何读取，如何更新

Memory

- claude code 在最近才开始支持 memory
- 模型并不知道什么是应该记住的，也很难正确提取关键点
- memory 如何存储，如何分层次存储，如何检索，如何读取，如何更新
- memory 对发散性思维的限制

Memory

- claude code 在最近才开始支持 memory
- 模型并不知道什么是应该记住的，也很难正确提取关键点
- memory 如何存储，如何分层次存储，如何检索，如何读取，如何更新
- memory 对发散性思维的限制
- 常见的两种解决方案

Memory

- claude code 在最近才开始支持 memory
- 模型并不知道什么是应该记住的，也很难正确提取关键点
- memory 如何存储，如何分层次存储，如何检索，如何读取，如何更新
- memory 对发散性思维的限制
- 常见的两种解决方案
 - memory.md

Memory

- claude code 在最近才开始支持 memory
- 模型并不知道什么是应该记住的，也很难正确提取关键点
- memory 如何存储，如何分层次存储，如何检索，如何读取，如何更新
- memory 对发散性思维的限制
- 常见的两种解决方案
 - memory.md
 - 结构化 json

OpenClaw

OpenClaw

- OpenClaw 的价值在于市场教育

OpenClaw

- OpenClaw 的价值在于市场教育
- 回到 2025 年初，cursor、claude code、manus 完成了对市场智能体的教育

OpenClaw

- OpenClaw 的价值在于市场教育
- 回到 2025 年初，cursor、claude code、manus 完成了对市场智能体的教育
- DeepSeek 对中国市场的教育

OpenClaw

- OpenClaw 的价值在于市场教育
- 回到 2025 年初，cursor、claude code、manus 完成了对市场智能体的教育
- DeepSeek 对中国市场的教育
- Claw 让大家放开了对全自主长存智能体的思维，也教育了市场的接受度

Q & A



KK



扫一扫上面的二维码图案，加我为朋友。

